Ulam Planner

Final Summative Exam – Project Output

Group Members:

Halasan, Paul Anderson

Lasala, Maria Georgianna

Lazaro, Gian Kylde

Usis, Troy Andre

OBJECTIVES

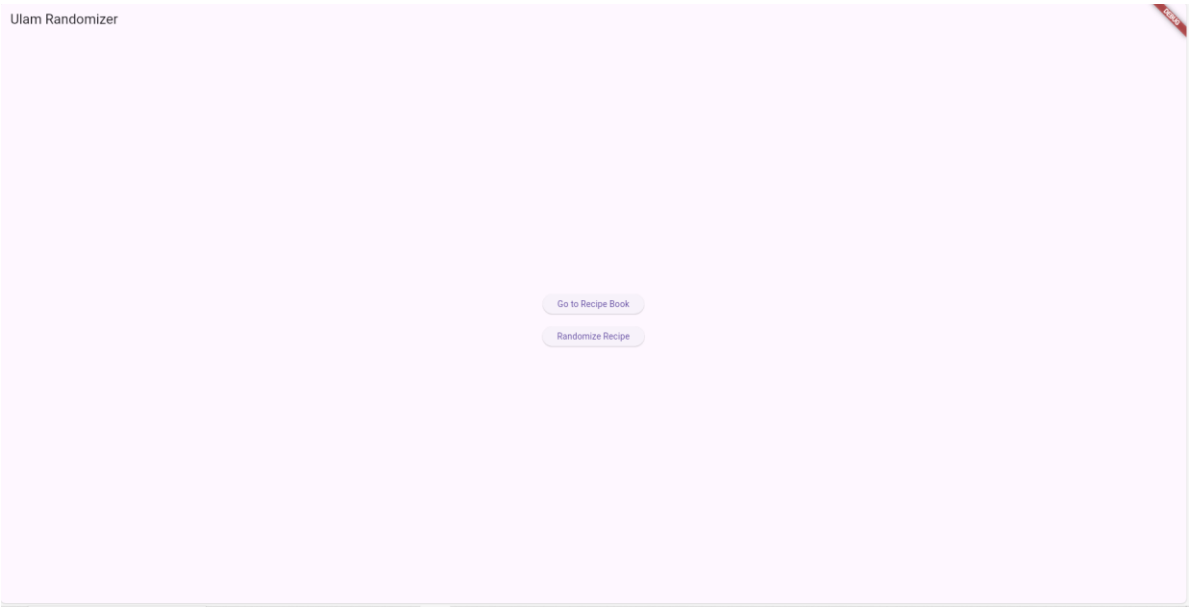
- The system will provide an easy-to-use feature for estimating ingredient amounts and associated costs based on varying serving sizes. Users can input the desired number of servings, and the system will automatically adjust ingredient quantities and calculate the total cost, simplifying meal planning and budgeting.
- Users will have the ability to customize recipes by selecting ingredients they already have on hand or substituting alternatives. The system will also allow users to set a target cost or dinner price, dynamically adjusting ingredient quantities and costs to align with their budget, ensuring cost-effective meal preparation.
- By automating ingredient calculations and cost adjustments, the system will reduce food wastage and encourage efficient use of resources. This will help users stay within their financial constraints while minimizing excess ingredients and leftovers, promoting a more sustainable and budget-conscious cooking experience.

SOLUTION

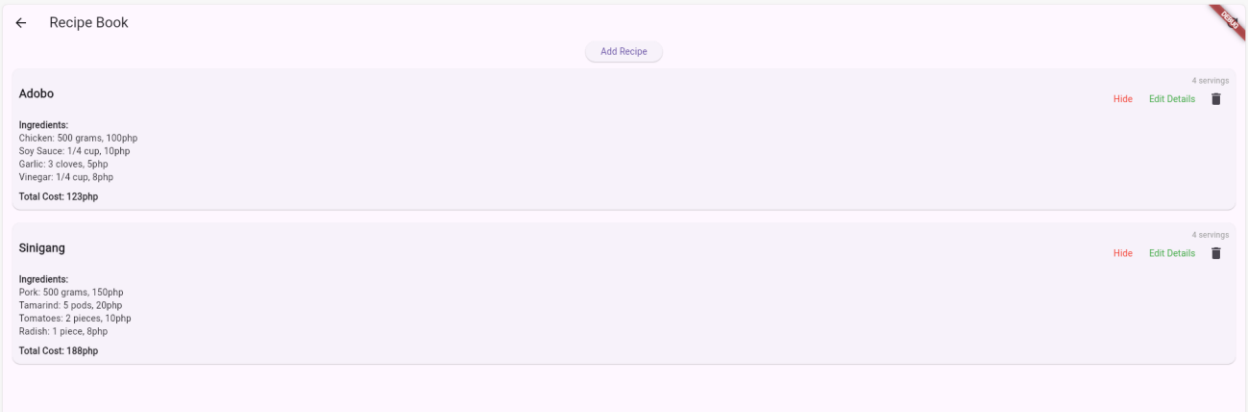
- To address the challenges of ingredient estimation, recipe customization, and budget management, the proposed solution focuses on integrating dynamic scaling, real-time cost calculation, and personalized adjustments into the system. For ingredient estimation, the system will use algorithms to proportionally adjust ingredient quantities based on user-input serving sizes. The user interface will feature intuitive input fields, allowing users to easily adjust serving sizes and immediately view updated quantities and costs.
- To enable recipe customization, the system will include a dedicated function that allows users to edit the ingredients of a specific dish or meal. This feature will provide users with the flexibility to modify recipes based on their preferences, dietary needs, or available ingredients. Users can directly adjust the type and quantity of each ingredient, tailoring the recipe to suit their requirements. Changes will be reflected dynamically, ensuring that users can instantly see the updated proportions and costs associated with their modifications.
- To reduce food wastage and promote efficient resource use, the system will automate ingredient calculations and cost adjustments to ensure precise meal planning. It will dynamically scale ingredient quantities based on the serving size specified by the user, allowing them to prepare only the necessary portions. This feature ensures users can make informed decisions while staying within their budget.

ACTUAL OUTPUT SCREENSHOTS

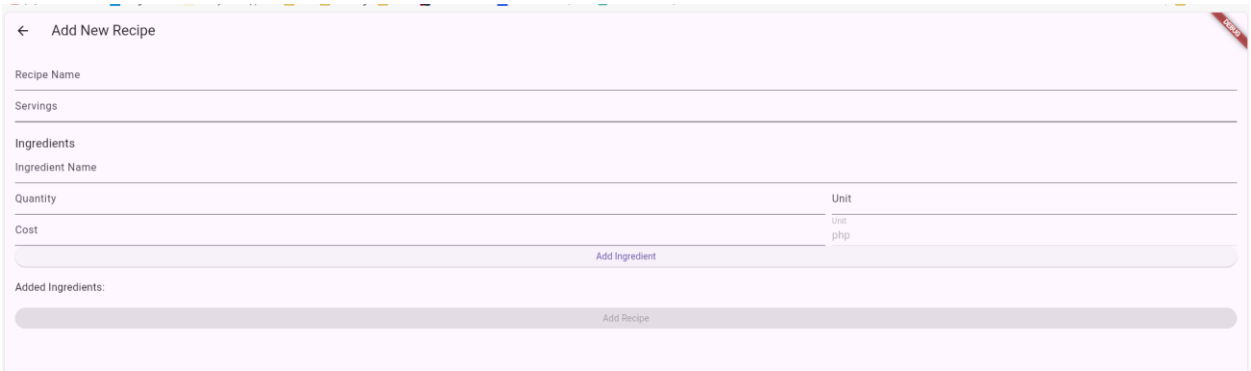
LANDING PAGE



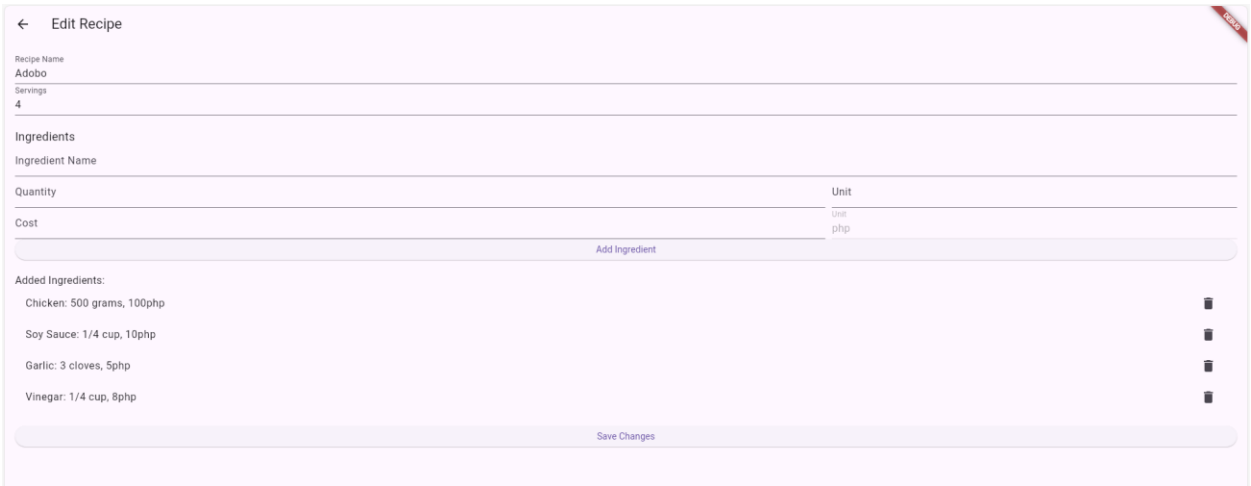
RECIPE BOOK



ADDING A NEW RECIPE



EDITING A RECIPE



RECIPE RANDOMIZER AND INPUT OF DESIRED SERVING

PROGRAM CODE

main.dart

```
import 'package:flutter/material.dart';
import 'recipe_manager.dart';
import 'recipe_book.dart';
import 'recipe_randomizer.dart';

void main() {
  runApp(UlamRandomizerApp());
}

class UlamRandomizerApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Ulam Randomizer',
      theme: ThemeData(primarySwatch: Colors.green),
      home: MainScreen(),
    );
  }
}

class MainScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Ulam Randomizer'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton(
              onPressed: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(builder: (context) => RecipeBookScreen()),
                );
              },
              child: Text('Go to Recipe Book'),
```

```
            ),
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                      builder: (context) => RecipeRandomizerScreen()),
                );
              },
              child: Text('Randomize Recipe'),
            ),
          ],
        ),
      ),
    );
  }
}
```

addrecipe.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart'; // For input formatters
import 'recipe_manager.dart';

class AddRecipeScreen extends StatefulWidget {
  @override
  _AddRecipeScreenState createState() => _AddRecipeScreenState();
}

class _AddRecipeScreenState extends State<AddRecipeScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _servingsController = TextEditingController();
  final _ingredientNameController = TextEditingController();
  final _ingredientQuantityNumberController = TextEditingController();
  final _ingredientQuantityUnitController = TextEditingController();
  final _ingredientCostController = TextEditingController();
  final _ingredientCostUnitController = TextEditingController(text: 'php');

  List<Ingredient> ingredients = [];

  bool get _isFormValid {
```

```
  return _nameController.text.isNotEmpty &&
      _servingsController.text.isNotEmpty &&
      int.tryParse(_servingsController.text) != null &&
      ingredients.isNotEmpty;
}

void _addRecipe() {
  if (_nameController.text.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Please enter a recipe name.'),
        backgroundColor: Colors.red,
      ),
    );
    return;
  }

  if (_servingsController.text.isEmpty ||
      int.tryParse(_servingsController.text) == null) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Please enter a valid number of servings.'),
        backgroundColor: Colors.red,
      ),
    );
    return;
  }

  if (ingredients.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Please add at least one ingredient.'),
        backgroundColor: Colors.red,
      ),
    );
    return;
  }

  for (var ingredient in ingredients) {
    if (ingredient.name.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Please complete the ingredient name.'),
          backgroundColor: Colors.red,
        ),
```

```
        );
        return;
      }
    }

    final newRecipe = Recipe(
      name: _nameController.text,
      servings: int.parse(_servingsController.text),
      ingredients: ingredients,
    );

    RecipeManager().addRecipe(newRecipe);

    Navigator.pop(context);
  }

  void _addIngredient() {
    if (_ingredientNameController.text.isEmpty ||
        _ingredientQuantityNumberController.text.isEmpty ||
        _ingredientQuantityUnitController.text.isEmpty ||
        _ingredientCostController.text.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Please fill in all fields for the ingredient.'),
          backgroundColor: Colors.red,
        ),
      );
    } else {
      final ingredient = Ingredient(
        name: _ingredientNameController.text,
        quantity:
            '${_ingredientQuantityNumberController.text}
${_ingredientQuantityUnitController.text}',
        cost:
            '${_ingredientCostController.text}${_ingredientCostUnitController.tex
t}',
      );

      setState(() {
        ingredients.add(ingredient);
      });

      _ingredientNameController.clear();
      _ingredientQuantityNumberController.clear();
      _ingredientQuantityUnitController.clear();
```

```
    _ingredientCostController.clear();
  }
}

void _removeIngredient(int index) {
  setState(() {
    ingredients.removeAt(index);
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Add New Recipe'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Form(
        key: _formKey,
        child: ListView(
          children: [
            TextFormField(
              controller: _nameController,
              decoration: InputDecoration(labelText: 'Recipe Name'),
              validator: (value) {
                if (value == null || value.isEmpty) {
                  return 'Please enter a recipe name';
                }
                return null;
              },
            ),
            TextFormField(
              controller: _servingsController,
              decoration: InputDecoration(labelText: 'Servings'),
              keyboardType: TextInputType.number,
              inputFormatters: [
                FilteringTextInputFormatter.digitsOnly,
              ],
              validator: (value) {
                if (value == null ||
                    value.isEmpty ||
                    int.tryParse(value) == null) {
                  return 'Please enter valid servings';
                }
```

```
            return null;
          },
        ),
        SizedBox(height: 20),
        Text('Ingredients', style: TextStyle(fontSize: 18)),
        TextFormField(
          controller: _ingredientNameController,
          decoration: InputDecoration(labelText: 'Ingredient Name'),
        ),
        Row(
          children: [
            Expanded(
              flex: 2,
              child: TextFormField(
                controller: _ingredientQuantityNumberController,
                decoration: InputDecoration(labelText: 'Quantity'),
                keyboardType:
                    TextInputType.numberWithOptions(decimal: true),
                inputFormatters: [
                  FilteringTextInputFormatter.allow(
                    RegExp(r'^([0-9]+(/[0-9]+)?|[0-9]+(\.[0-9]+)?)?$'),
                  ),
                ],
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'Please enter a valid quantity';
                  }
                  return null;
                },
              ),
            ),
            SizedBox(width: 10),
            Expanded(
              flex: 1,
              child: TextFormField(
                controller: _ingredientQuantityUnitController,
                decoration: InputDecoration(labelText: 'Unit'),
              ),
            ),
          ],
        ),
        Row(
          children: [
            Expanded(
              flex: 2,
```

```
                    child: TextFormField(
                      controller: _ingredientCostController,
                      decoration: InputDecoration(labelText: 'Cost'),
                      keyboardType:
                          TextInputType.numberWithOptions(decimal: true),
                      inputFormatters: [
                        FilteringTextInputFormatter.allow(RegExp(
                            r'^[0-9]*\.?[0-9]*$')), // Allow only digits and a
single decimal point
                      ],
                      validator: (value) {
                        if (value == null || value.isEmpty) {
                          return 'Please enter a valid cost';
                        }
                        return null;
                      },
                    ),
                  ),
                  SizedBox(width: 10),
                  Expanded(
                    flex: 1,
                    child: TextFormField(
                      controller: _ingredientCostUnitController,
                      decoration: InputDecoration(labelText: 'Unit'),
                      enabled: false,
                    ),
                  ),
                ],
              ),
              ElevatedButton(
                onPressed: _addIngredient,
                child: Text('Add Ingredient'),
              ),
              SizedBox(height: 20),
              Text('Added Ingredients:', style: TextStyle(fontSize: 16)),
              for (var i = 0; i < ingredients.length; i++)
                ListTile(
                  title: Text(
                      '${ingredients[i].name}: ${ingredients[i].quantity},
${ingredients[i].cost}'),
                  trailing: IconButton(
                    icon: Icon(Icons.delete),
                    onPressed: () => _removeIngredient(i),
                  ),
                ),
```

```
              SizedBox(height: 20),
              ElevatedButton(
                onPressed: _isFormValid ? _addRecipe : null,
                child: Text('Add Recipe'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

editrecipe.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart'; // For input formatters
import 'recipe_manager.dart';

class EditRecipeScreen extends StatefulWidget {
  final Recipe recipe;

  EditRecipeScreen({required this.recipe});

  @override
  _EditRecipeScreenState createState() => _EditRecipeScreenState();
}

class _EditRecipeScreenState extends State<EditRecipeScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _servingsController = TextEditingController();
  final _ingredientNameController = TextEditingController();
  final _ingredientQuantityNumberController = TextEditingController();
  final _ingredientQuantityUnitController = TextEditingController();
  final _ingredientCostController = TextEditingController();
  final _ingredientCostUnitController = TextEditingController(text: 'php');

  List<Ingredient> ingredients = [];

  bool get _isFormValid {
    return _nameController.text.isNotEmpty &&
        _servingsController.text.isNotEmpty &&
```

```
      int.tryParse(_servingsController.text) != null &&
      ingredients.isNotEmpty;
}

@override
void initState() {
  super.initState();

  // Prepopulate fields with existing recipe data
  _nameController.text = widget.recipe.name;
  _servingsController.text = widget.recipe.servings.toString();
  ingredients = List.from(widget.recipe.ingredients);

  // Prepare controllers for ingredient editing (if necessary)
  _ingredientNameController.clear();
  _ingredientQuantityNumberController.clear();
  _ingredientQuantityUnitController.clear();
  _ingredientCostController.clear();
}

void _editRecipe() {
  if (_nameController.text.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Please enter a recipe name.'),
        backgroundColor: Colors.red,
      ),
    );
    return;
  }

  if (_servingsController.text.isEmpty ||
      int.tryParse(_servingsController.text) == null) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Please enter a valid number of servings.'),
        backgroundColor: Colors.red,
      ),
    );
    return;
  }

  if (ingredients.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
```

```
        content: Text('Please add at least one ingredient.'),
        backgroundColor: Colors.red,
      ),
    );
    return;
  }

  for (var ingredient in ingredients) {
    if (ingredient.name.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Please complete the ingredient name.'),
          backgroundColor: Colors.red,
        ),
      );
      return;
    }
  }

  final updatedRecipe = Recipe(
    name: _nameController.text,
    servings: int.parse(_servingsController.text),
    ingredients: ingredients,
  );

  // Pass the original recipe name (widget.recipe.name) and the updated recipe
  RecipeManager().updateRecipe(widget.recipe.name, updatedRecipe);

  Navigator.pop(context);
}

void _addIngredient() {
  if (_ingredientNameController.text.isEmpty ||
      _ingredientQuantityNumberController.text.isEmpty ||
      _ingredientQuantityUnitController.text.isEmpty ||
      _ingredientCostController.text.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Please fill in all fields for the ingredient.'),
        backgroundColor: Colors.red,
      ),
    );
  } else {
    final ingredient = Ingredient(
      name: _ingredientNameController.text,
```

```
        quantity:
            '${_ingredientQuantityNumberController.text}
${_ingredientQuantityUnitController.text}',
        cost:
            '${_ingredientCostController.text}${_ingredientCostUnitController.tex
t}',
      );

      setState(() {
        ingredients.add(ingredient);
      });

      _ingredientNameController.clear();
      _ingredientQuantityNumberController.clear();
      _ingredientQuantityUnitController.clear();
      _ingredientCostController.clear();
    }
  }

  void _removeIngredient(int index) {
    setState(() {
      ingredients.removeAt(index);
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Edit Recipe'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Form(
          key: _formKey,
          child: ListView(
            children: [
              TextFormField(
                controller: _nameController,
                decoration: InputDecoration(labelText: 'Recipe Name'),
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'Please enter a recipe name';
                  }
                  return null;
```

```
            },
          ),
          TextFormField(
            controller: _servingsController,
            decoration: InputDecoration(labelText: 'Servings'),
            keyboardType: TextInputType.number,
            inputFormatters: [
              FilteringTextInputFormatter.digitsOnly,
            ],
            validator: (value) {
              if (value == null ||
                  value.isEmpty ||
                  int.tryParse(value) == null) {
                return 'Please enter valid servings';
              }
              return null;
            },
          ),
          SizedBox(height: 20),
          Text('Ingredients', style: TextStyle(fontSize: 18)),
          TextFormField(
            controller: _ingredientNameController,
            decoration: InputDecoration(labelText: 'Ingredient Name'),
          ),
          Row(
            children: [
              Expanded(
                flex: 2,
                child: TextFormField(
                  controller: _ingredientQuantityNumberController,
                  decoration: InputDecoration(labelText: 'Quantity'),
                  keyboardType: TextInputType
                      .text, // Use Text type to allow all characters
                  inputFormatters: [
                    FilteringTextInputFormatter.allow(
                      RegExp(
                          r'[0-9/.\s]'), // Allow digits, '/', '.', and space
                    ),
                  ],
                  validator: (value) {
                    if (value == null || value.isEmpty) {
                      return 'Please enter a valid quantity';
                    }
                    return null;
                  },
```

```
                    ),
                  ),
                  SizedBox(width: 10),
                  Expanded(
                    flex: 1,
                    child: TextFormField(
                      controller: _ingredientQuantityUnitController,
                      decoration: InputDecoration(labelText: 'Unit'),
                    ),
                  ),
                ],
              ),
              Row(
                children: [
                  Expanded(
                    flex: 2,
                    child: TextFormField(
                      controller: _ingredientCostController,
                      decoration: InputDecoration(labelText: 'Cost'),
                      keyboardType:
                          TextInputType.numberWithOptions(decimal: true),
                      inputFormatters: [
                        FilteringTextInputFormatter.allow(RegExp(
                            r'^[0-9]*\.?[0-9]*$')), // Allow only digits and a
single decimal point
                      ],
                      validator: (value) {
                        if (value == null || value.isEmpty) {
                          return 'Please enter a valid quantity';
                        }
                        return null;
                      },
                    ),
                  ),
                  SizedBox(width: 10),
                  Expanded(
                    flex: 1,
                    child: TextFormField(
                      controller: _ingredientCostUnitController,
                      decoration: InputDecoration(labelText: 'Unit'),
                      enabled: false,
                    ),
                  ),
                ],
              ),
```

```
              ElevatedButton(
                onPressed: _addIngredient,
                child: Text('Add Ingredient'),
              ),
              SizedBox(height: 20),
              Text('Added Ingredients:', style: TextStyle(fontSize: 16)),
              for (var i = 0; i < ingredients.length; i++)
                ListTile(
                  title: Text(
                      '${ingredients[i].name}: ${ingredients[i].quantity},
${ingredients[i].cost}'),
                  trailing: IconButton(
                    icon: Icon(Icons.delete),
                    onPressed: () => _removeIngredient(i),
                  ),
                ),
              SizedBox(height: 20),
              ElevatedButton(
                onPressed: _isFormValid ? _editRecipe : null,
                child: Text('Save Changes'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

recipe_book.dart

```
import 'package:flutter/material.dart';
import 'recipe_manager.dart';
import 'add_recipe.dart'; // Import AddRecipeScreen
import 'edit_recipe.dart'; // Import EditRecipeScreen

class RecipeBookScreen extends StatefulWidget {
  @override
  _RecipeBookScreenState createState() => _RecipeBookScreenState();
}

class _RecipeBookScreenState extends State<RecipeBookScreen> {
  Map<int, bool> expandedStates = {};
```

```
double getTotalCost(Recipe recipe) {
  double totalCost = 0.0;
  for (var ingredient in recipe.ingredients) {
    totalCost += double.tryParse(ingredient.cost
            .replaceAll('php', '')
            .replaceAll('P', '')
            .trim()) ??
        0.0;
  }
  return totalCost;
}

void removeRecipe(int index) {
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return AlertDialog(
        title: Text("Are you sure?"),
        content: Text("Do you really want to remove this recipe?"),
        actions: [
          TextButton(
            onPressed: () {
              setState(() {
                RecipeManager().recipes.removeAt(index);
              });
              Navigator.of(context).pop();
            },
            child: Text("Yes"),
          ),
          TextButton(
            onPressed: () {
              Navigator.of(context).pop();
            },
            child: Text("No"),
          ),
        ],
      );
    },
  );
}

void refreshRecipes() {
  setState(() {
    // Trigger a rebuild to fetch updated recipes
```

```
  });
}

@override
Widget build(BuildContext context) {
  final recipeManager = RecipeManager();

  return Scaffold(
    appBar: AppBar(
      title: Text('Recipe Book'),
      actions: [
        IconButton(
          icon: Icon(Icons.refresh),
          onPressed: refreshRecipes, // Call refreshRecipes method
        ),
      ],
    ),
    body: Column(
      children: [
        ElevatedButton(
          onPressed: () async {
            await Navigator.push(
              context,
              MaterialPageRoute(
                  builder: (context) =>
                      AddRecipeScreen()), // Navigate to AddRecipeScreen
            );
            refreshRecipes(); // Refresh the list when coming back
          },
          child: Text('Add Recipe'),
        ),
        Expanded(
          child: recipeManager.recipes.isEmpty
              ? Center(
                  child: Text(
                    'Recipe book is empty.',
                    style: TextStyle(fontSize: 18, color: Colors.grey),
                  ),
                )
              : ListView.builder(
                  itemCount: recipeManager.recipes.length,
                  itemBuilder: (context, index) {
                    final recipe = recipeManager.recipes[index];

                    return Card(
```

```
margin:
    EdgeInsets.symmetric(vertical: 10, horizontal: 15),
child: Padding(
  padding: EdgeInsets.all(10),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Row(
        mainAxisAlignment:
            MainAxisAlignment.spaceBetween,
        children: [
          Text(
            recipe.name,
            style: TextStyle(
                fontWeight: FontWeight.bold,
                fontSize: 18),
          ),
          Column(
            crossAxisAlignment: CrossAxisAlignment.end,
            children: [
              Text(
                '${recipe.servings} servings',
                style: TextStyle(
                    fontSize: 12, color: Colors.grey),
              ),
              Row(
                children: [
                  TextButton(
                    onPressed: () {
                      setState(() {
                        expandedStates[index] =
                            !(expandedStates[index] ??
                                false);
                      });
                    },
                    child: Text(
                      expandedStates[index] ?? false
                          ? 'Hide'
                          : 'Details',
                      style: TextStyle(
                          color:
                              expandedStates[index] ??
                                      false
                                  ? Colors.red
                                  : Colors.blue),
```

```
                                        ),
                                      ),
                                      // Edit Button
                                      TextButton(
                                        onPressed: () async {
                                          await Navigator.push(
                                            context,
                                            MaterialPageRoute(
                                              builder: (context) =>
                                                  EditRecipeScreen(
                                                      recipe:
                                                          recipe), //
Navigate to EditRecipeScreen
                                            ),
                                          );
                                          refreshRecipes(); // Refresh the
list after editing
                                        },
                                        child: Text('Edit Details',
                                            style: TextStyle(
                                                color: Colors.green)),
                                      ),
                                      // Delete Button
                                      IconButton(
                                        icon: Icon(Icons.delete),
                                        onPressed: () =>
                                            removeRecipe(index),
                                      ),
                                    ],
                                  ),
                                ],
                              ),
                            ],
                          ),
                          if (expandedStates[index] ?? false)
                            Column(
                              crossAxisAlignment: CrossAxisAlignment.start,
                              children: [
                                SizedBox(height: 10),
                                Text('Ingredients:',
                                    style: TextStyle(
                                        fontWeight: FontWeight.bold)),
                                for (var ingredient in recipe.ingredients)
                                  Text(
```

```
                                        '${ingredient.name}:
${ingredient.quantity}, ${ingredient.cost}',
                                        style: TextStyle(fontSize: 14)),
                                    SizedBox(height: 10),
                                    Text(
                                        'Total Cost: ${getTotalCost(recipe)}php',
                                        style: TextStyle(
                                            fontWeight: FontWeight.bold)),
                                  ],
                                ),
                              ],
                            ),
                          ),
                        );
                      },
                    ),
                  ),
                ],
              ),
            );
          }
}
```

recipe_manager.dart

```
// Ingredient Class
class Ingredient {
  final String name;
  final String quantity;
  final String cost;

  Ingredient({
    required this.name,
    required this.quantity,
    required this.cost,
  });
}

// Recipe Class
class Recipe {
  final String name;
  final List<Ingredient> ingredients;
```

```
final int servings;

Recipe({
  required this.name,
  required this.ingredients,
  required this.servings,
});

double parseFraction(String input) {
  if (input.contains('/')) {
    var parts = input.split('/');
    if (parts.length == 2) {
      double numerator = double.tryParse(parts[0]) ?? 0;
      double denominator = double.tryParse(parts[1]) ?? 1;
      return numerator / denominator;
    }
  }
  return double.tryParse(input) ?? 0;
}

List<Ingredient> scaleIngredients(int desiredServings) {
  double scaleFactor = desiredServings / servings;
  return ingredients.map((ingredient) {
    List<String> parts = ingredient.quantity.split(' ');
    String quantityStr = parts.first;
    String unit = parts.length > 1 ? parts.sublist(1).join(' ') : '';

    double quantity = parseFraction(quantityStr);
    double scaledQuantity = quantity * scaleFactor;

    double cost =
        double.tryParse(ingredient.cost.replaceAll(RegExp(r'[^0-9.]'), '')) ??
            0;
    double scaledCost = cost * scaleFactor;

    return Ingredient(
      name: ingredient.name,
      quantity: '${scaledQuantity.toStringAsFixed(2)} $unit',
      cost: '${scaledCost.toStringAsFixed(2)}php',
    );
  }).toList();
}

double calculateTotalCost(int desiredServings) {
  double scaleFactor = desiredServings / servings;
```

```
    return ingredients.fold(0, (total, ingredient) {
      double cost =
          double.tryParse(ingredient.cost.replaceAll(RegExp(r'[^0-9.]'), '')) ??
              0;
      return total + (cost * scaleFactor);
    });
  }
}

// RecipeManager Class
class RecipeManager {
  // Singleton Instance
  static final RecipeManager _instance = RecipeManager._internal();

  // Factory Constructor
  factory RecipeManager() => _instance;

  // Private Constructor
  RecipeManager._internal();

  // List of Recipes
  final List<Recipe> recipes = [
    Recipe(
      name: 'Adobo',
      ingredients: [
        Ingredient(name: 'Chicken', quantity: '500 grams', cost: '100php'),
        Ingredient(name: 'Soy Sauce', quantity: '1/4 cup', cost: '10php'),
        Ingredient(name: 'Garlic', quantity: '3 cloves', cost: '5php'),
        Ingredient(name: 'Vinegar', quantity: '1/4 cup', cost: '8php'),
      ],
      servings: 4,
    ),
    Recipe(
      name: 'Sinigang',
      ingredients: [
        Ingredient(name: 'Pork', quantity: '500 grams', cost: '150php'),
        Ingredient(name: 'Tamarind', quantity: '5 pods', cost: '20php'),
        Ingredient(name: 'Tomatoes', quantity: '2 pieces', cost: '10php'),
        Ingredient(name: 'Radish', quantity: '1 piece', cost: '8php'),
      ],
      servings: 4,
    ),
    // Add more recipes here...
  ];
```

```
  // Add Recipe Method
  void addRecipe(Recipe recipe) {
    recipes.add(recipe);
  }

  // Update Recipe Method
  void updateRecipe(String name, Recipe updatedRecipe) {
    // Find the index of the recipe to update by name
    int index = recipes.indexWhere((recipe) => recipe.name == name);

    if (index != -1) {
      // Recipe found, update it
      recipes[index] = updatedRecipe;
      print('Recipe "$name" updated successfully!');
    } else {
      // Recipe not found
      print('Recipe "$name" not found!');
    }
  }
}
```

recipe_randomizer.dart

```
import 'package:flutter/material.dart';
import 'recipe_manager.dart';

class RecipeRandomizerScreen extends StatefulWidget {
  @override
  _RecipeRandomizerScreenState createState() => _RecipeRandomizerScreenState();
}

class _RecipeRandomizerScreenState extends State<RecipeRandomizerScreen> {
  final recipeManager = RecipeManager();
  final recipes = RecipeManager().recipes;
  int desiredServings = 1;
  Recipe? randomRecipe;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Recipe Randomizer'),
```

```
    ),
    body: Center(
      child: recipes.isEmpty
          ? Text(
              'No recipes available!',
              style: TextStyle(fontSize: 18),
            )
          : Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                ElevatedButton(
                  onPressed: () {
                    setState(() {
                      randomRecipe = (recipes..shuffle()).first;
                    });
                  },
                  child: Text('Randomize Recipe'),
                ),
                if (randomRecipe != null) ...[
                  SizedBox(height: 20),
                  Text(
                    'Random Recipe: ${randomRecipe!.name}',
                    style:
                        TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                  ),
                  SizedBox(height: 20),
                  TextField(
                    decoration: InputDecoration(
                      labelText: 'Desired Servings',
                      border: OutlineInputBorder(),
                    ),
                    keyboardType: TextInputType.number,
                    onChanged: (value) {
                      setState(() {
                        desiredServings = int.tryParse(value) ?? 1;
                      });
                    },
                  ),
                  SizedBox(height: 20),
                  Text(
                    'Ingredients for $desiredServings servings:',
                    style:
                        TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
                  ),
                  ...randomRecipe!
```

```
                        .scaleIngredients(desiredServings)
                        .map((ingredient) {
                      return Text(
                          '${ingredient.name}: ${ingredient.quantity}
(${ingredient.cost})');
                    }).toList(),
                    SizedBox(height: 20),
                    Text(
                      'Total Cost:
${randomRecipe!.calculateTotalCost(desiredServings).toStringAsFixed(2)}php',
                      style:
                          TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
                    ),
                  ],
                ],
              ),
      ),
    );
  }
}
```